TASK 6: PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

STUDENT DETAILS:

Name: Divya. V
Roll No: E25AI111
Department: B.Sc Computer Science With AI

SCENARIO

The Artisan Crumb Bakery & Pastry Management System wants to collect customer feedback about purchased bakery items. Customers can provide ratings and reviews after completing their bakery orders. The system should store customer opinions, analyze bakery item performance, and identify highly rated bakery products. Students will create a Product Review and Rating Module connected with the existing Customer and Product modules.

1. Create Review and Rating Tables

```
SQL> CREATE TABLE Review (
  2      Review_ID    VARCHAR2(7),
  3      Customer_ID  VARCHAR2(7),
  4      Product_ID   VARCHAR2(7),
  5      Rating       NUMBER(1),
  6      Review_Text  VARCHAR2(250),
  7      Review_Date  DATE,
  8      CONSTRAINT PK_Review PRIMARY KEY (Review_ID),
  9      CONSTRAINT FK_Review_Customer FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
 10      CONSTRAINT FK_Review_Product FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
 11  );

Table created.
```

```
SQL> desc Review
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 REVIEW_ID                                 NOT NULL VARCHAR2(7)
 CUSTOMER_ID                                        VARCHAR2(7)
 PRODUCT_ID                                         VARCHAR2(7)
 RATING                                             NUMBER(1)
 REVIEW_TEXT                                        VARCHAR2(250)
 REVIEW_DATE                                        DATE
```

Store Customer Feedback and Ratings

Tasks:

1. Add reviews for purchased products

```
SQL> INSERT INTO Review VALUES ('REV001','C001','P001',5, 'Excellent Chocolate Fudge Cake', DATE '2026-01-20');

1 row created.

SQL> INSERT INTO Review VALUES ('REV002','C002','P002',4, 'Fresh and tasty Sourdough Bread', DATE '2026-01-22');

1 row created.

SQL> INSERT INTO Review VALUES ('REV003','C003','P003',5, 'Delicious Red Velvet Pastry', DATE '2026-02-01');

1 row created.

SQL> INSERT INTO Review VALUES ('REV004','C004','P004',4, 'Very good Croissants', DATE '2026-02-05');

1 row created.

SQL> INSERT INTO Review VALUES ('REV005','C005','P005',5, 'Excellent Black Forest Cake', DATE '2026-02-10');

1 row created.
```

```
SQL> SELECT * FROM Review;

REVIEW_ CUSTOME PRODUCT     RATING
------- ------- ------- ----------
REVIEW_TEXT
--------------------------------------------
REVIEW_DA
---------
REV001  C001    P001             5
Excellent Chocolate Fudge Cake
20-JAN-26

REV002  C002    P002             4
Fresh and tasty Sourdough Bread
22-JAN-26

REVIEW_ CUSTOME PRODUCT     RATING
------- ------- ------- ----------
REVIEW_TEXT
--------------------------------------------
REVIEW_DA
---------

REV003  C003    P003             5
Delicious Red Velvet Pastry
01-FEB-26

REV004  C004    P004             4
Very good Croissants

REVIEW_ CUSTOME PRODUCT     RATING
------- ------- ------- ----------
REVIEW_TEXT
--------------------------------------------
REVIEW_DA
---------
05-FEB-26

REV005  C005    P005             5
Excellent Black Forest Cake
10-FEB-26
```

2. Store customer ratings from 1 to 5 stars

Ensure this by adding Constraint

```
SQL> ALTER TABLE Review
  2  ADD CONSTRAINT CHK_Rating
  3  CHECK (Rating >= 1 AND Rating <= 5);

Table altered.
```

3. Update review comments if required

```
SQL> UPDATE Review
  2  SET Review_Text = 'Good quality bakery item with fresh ingredients'
  3  WHERE Review_ID = 'REV004';

1 row updated.
```

4. Remove inappropriate or invalid reviews

```
SQL> DELETE FROM Review
  2  WHERE Review_ID = 'REV005';

1 row deleted.
```

4.Retrieve Product Review Details

Write SQL queries to:

1. Display all reviews for a product

```
SQL> SELECT *
  2  FROM Review
  3  WHERE Product_ID = 'P001';

REVIEW_ CUSTOME PRODUCT     RATING
------- ------- ------- ----------
REVIEW_TEXT
--------------------------------------------------
REVIEW_DA
---------
REV001  C001    P001             5
Excellent Chocolate Fudge Cake
20-JAN-26
```

2. Display customer name with their reviews

```
SQL> SELECT
  2  C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  3  R.Review_ID,
  4  R.Product_ID,
  5  R.Rating,
  6  R.Review_Text
  7  FROM Customer C
  8  JOIN Review R
  9  ON C.Customer_ID = R.Customer_ID;

CUSTOMER_NAME                              REVIEW_ PRODUCT     RATING
------------------------------------------ ------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
John Doe                                   REV001  P001             5
Excellent Chocolate Fudge Cake

Jane Smith                                 REV002  P002             4
Fresh and tasty Sourdough Bread

Alice Brown                                REV003  P003             5
Delicious Red Velvet Pastry


CUSTOMER_NAME                              REVIEW_ PRODUCT     RATING
------------------------------------------ ------- ------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
Bob White                                  REV004  P004             4
Good quality bakery item with fresh ingredients
```

3. Find products having maximum reviews

```
SQL> SELECT
  2  P.Product_ID,
  3  P.Title,
  4  COUNT(R.Review_ID) AS Review_Count
  5  FROM Product P
  6  JOIN Review R
  7  ON P.Product_ID = R.Product_ID
  8  GROUP BY P.Product_ID, P.Title
  9  ORDER BY Review_Count DESC;

PRODUCT
-------
TITLE
----------------------------------------
REVIEW_COUNT
------------
P001
Chocolate Fudge Cake
           1

P004
Croissants
           1

PRODUCT
-------
TITLE
----------------------------------------
REVIEW_COUNT
------------

P003
Red Velvet Pastry
           1

P002
Sourdough Bread

PRODUCT
-------
TITLE
----------------------------------------
REVIEW_COUNT
------------
           1
```

4. Display recent customer feedback

```
SQL> SELECT *
  2  FROM
  3  (
  4  SELECT
  5  C.First_Name || ' ' || C.Last_Name AS Customer_Name,
  6  P.Title,
  7  R.Rating,
  8  R.Review_Text,
  9  R.Review_Date
 10  FROM Review R
 11  JOIN Customer C
 12  ON R.Customer_ID = C.Customer_ID
 13  JOIN Product P
 14  ON R.Product_ID = P.Product_ID
 15  ORDER BY R.Review_Date DESC
 16  )
 17  WHERE ROWNUM <= 5;

CUSTOMER_NAME
----------------------------------------
TITLE
------------------------------------------------------------
    RATING
----------
REVIEW_TEXT
------------------------------------------------------------
REVIEW_DA
---------
Bob White
Croissants
         4

CUSTOMER_NAME
----------------------------------------
TITLE
------------------------------------------------------------
    RATING
----------
REVIEW_TEXT
------------------------------------------------------------
REVIEW_DA
---------
Good quality bakery item with fresh ingredients
05-FEB-26
```

```
CUSTOMER_NAME
----------------------------------------
TITLE
----------------------------------------
    RATING
----------
REVIEW_TEXT
----------------------------------------
REVIEW_DA
---------
Alice Brown
Red Velvet Pastry
         5

CUSTOMER_NAME
----------------------------------------
TITLE
----------------------------------------
    RATING
----------
REVIEW_TEXT
----------------------------------------
REVIEW_DA
---------
Delicious Red Velvet Pastry
01-FEB-26


CUSTOMER_NAME
----------------------------------------
TITLE
----------------------------------------
    RATING
----------
REVIEW_TEXT
----------------------------------------
REVIEW_DA
---------
Jane Smith
Sourdough Bread
         4

CUSTOMER_NAME
----------------------------------------
TITLE
----------------------------------------
    RATING
----------
REVIEW_TEXT
----------------------------------------
REVIEW_DA
---------
Fresh and tasty Sourdough Bread
22-JAN-26
```

```
CUSTOMER_NAME
----------------------------------------
TITLE
----------------------------------------
    RATING
----------
REVIEW_TEXT
----------------------------------------
REVIEW_DA
---------
John Doe
Chocolate Fudge Cake
         5

CUSTOMER_NAME
----------------------------------------
TITLE
----------------------------------------
    RATING
----------
REVIEW_TEXT
----------------------------------------
REVIEW_DA
---------
Excellent Chocolate Fudge Cake
20-JAN-26
```

5. Retrieve reviews with ratings above 4

```
SQL> SELECT *
  2  FROM Review
  3  WHERE Rating > 4;

REVIEW_ CUSTOME PRODUCT     RATING
------- ------- ------- ----------
REVIEW_TEXT
----------------------------------------
REVIEW_DA
---------
REV001  C001    P001             5
Excellent Chocolate Fudge Cake
20-JAN-26

REV003  C003    P003             5
Delicious Red Velvet Pastry
01-FEB-26

REVIEW_ CUSTOME PRODUCT     RATING
------- ------- ------- ----------
REVIEW_TEXT
----------------------------------------
REVIEW_DA
---------
```

Example Query

5. Calculate Average Product Ratings

Use aggregate functions to analyze ratings.

Tasks:

1.Calculate average rating for each product

```
SQL> SELECT
  2  P.Title,
  3  AVG(R.Rating) AS Average_Rating
  4  FROM Product P
  5  JOIN Review R
  6  ON P.Product_ID = R.Product_ID
  7  GROUP BY P.Title;

TITLE
--------------------------------------
AVERAGE_RATING
--------------
Sourdough Bread
             4

Croissants
             4

Chocolate Fudge Cake
             5


TITLE
--------------------------------------
AVERAGE_RATING
--------------
Red Velvet Pastry
             5
```

2. Count the number of reviews per product

```
SQL> SELECT
  2  P.Title,
  3  COUNT(R.Rating) AS Count_Rating
  4  FROM Product P
  5  JOIN Review R
  6  ON P.Product_ID = R.Product_ID
  7  GROUP BY P.Title;

TITLE
----------------------------------------
COUNT_RATING
------------
Sourdough Bread
           1

Croissants
           1

Chocolate Fudge Cake
           1


TITLE
----------------------------------------
COUNT_RATING
------------
Red Velvet Pastry
           1
```

3. Find highest-rated products

```
SQL> SELECT
  2  P.Product_ID,
  3  P.Title,
  4  ROUND(AVG(R.Rating),2) AS Average_Rating
  5  FROM Product P
  6  JOIN Review R
  7  ON P.Product_ID = R.Product_ID
  8  GROUP BY P.Product_ID, P.Title
  9  HAVING AVG(R.Rating) =
 10  (
 11  SELECT MAX(Avg_Rating)
 12  FROM
 13  (
 14  SELECT AVG(Rating) AS Avg_Rating
 15  FROM Review
 16  GROUP BY Product_ID
 17  )
 18  );

PRODUCT
-------
TITLE
--------------------------------------------
AVERAGE_RATING
--------------
P001
Chocolate Fudge Cake
             5

P003
Red Velvet Pastry
             5

PRODUCT
-------
TITLE
--------------------------------------------
AVERAGE_RATING
--------------
```

4. Identify products with average rating above 4

```
SQL> SELECT
  2  P.Product_ID,
  3  P.Title,
  4  ROUND(AVG(R.Rating),2) AS Average_Rating
  5  FROM Product P
  6  JOIN Review R
  7  ON P.Product_ID = R.Product_ID
  8  GROUP BY P.Product_ID, P.Title
  9  HAVING AVG(R.Rating) > 4
 10  ORDER BY Average_Rating DESC;

PRODUCT
-------
TITLE
--------------------------------------------------
AVERAGE_RATING
--------------
P003
Red Velvet Pastry
             5

P001
Chocolate Fudge Cake
             5

PRODUCT
-------
TITLE
--------------------------------------------------
AVERAGE_RATING
--------------
```

6. Identify Highly Rated Products

Generate product rating reports.

Report 1: Product Rating Analysis

Display:

- Product Name
- Number of Reviews
- Average Rating

```
SQL> SELECT
  2  P.Title,
  3  COUNT(R.Review_ID) AS Number_Of_Reviews,
  4  ROUND(AVG(R.Rating),2) AS Average_Rating
  5  FROM Product P
  6  LEFT JOIN Review R
  7  ON P.Product_ID = R.Product_ID
  8  GROUP BY P.Product_ID, P.Title
  9  ORDER BY Average_Rating DESC NULLS LAST;

TITLE
--------------------------------------------------
NUMBER_OF_REVIEWS AVERAGE_RATING
----------------- --------------
Chocolate Fudge Cake
                1              5

Red Velvet Pastry
                1              5

Croissants
                1              4


TITLE
--------------------------------------------------
NUMBER_OF_REVIEWS AVERAGE_RATING
----------------- --------------
Sourdough Bread
                1              4

Black Forest Cake
                0
```

Report 2: Customer Feedback Analysis

Most Reviewed Products

```
SQL> SELECT
  2  P.Title,
  3  COUNT(R.Review_ID) AS Review_Count
  4  FROM Product P
  5  JOIN Review R
  6  ON P.Product_ID = R.Product_ID
  7  GROUP BY P.Product_ID, P.Title
  8  ORDER BY Review_Count DESC;

TITLE
----------------------------------------
REVIEW_COUNT
------------
Chocolate Fudge Cake
           1

Croissants
           1

Red Velvet Pastry
           1


TITLE
----------------------------------------
REVIEW_COUNT
------------
Sourdough Bread
           1
```

Highly Rated Products

```
SQL> SELECT
  2  P.Title,
  3  ROUND(AVG(R.Rating),2) AS Average_Rating
  4  FROM Product P
  5  JOIN Review R
  6  ON P.Product_ID = R.Product_ID
  7  GROUP BY P.Product_ID, P.Title
  8  HAVING AVG(R.Rating) > 4
  9  ORDER BY Average_Rating DESC;

TITLE
--------------------------------------------------
AVERAGE_RATING
--------------
Red Velvet Pastry
             5

Chocolate Fudge Cake
             5
```

Products Requiring Improvement

```
SQL> SELECT
  2  P.Title,
  3  COUNT(R.Review_ID) AS Review_Count,
  4  ROUND(AVG(R.Rating),2) AS Average_Rating
  5  FROM Product P
  6  JOIN Review R
  7  ON P.Product_ID = R.Product_ID
  8  GROUP BY P.Product_ID, P.Title
  9  HAVING AVG(R.Rating) < 3
 10  ORDER BY Average_Rating;

no rows selected
```

Report 3: Rating Distribution

Number of 5-Star Ratings

```
SQL> SELECT COUNT(*) AS Five_Star_Ratings
  2  FROM Review
  3  WHERE Rating = 5;

FIVE_STAR_RATINGS
-----------------
                2
```

Number of 4-Star Ratings

```
SQL> SELECT COUNT(*) AS Four_Star_Ratings
  2  FROM Review
  3  WHERE Rating = 4;

FOUR_STAR_RATINGS
-----------------
                2
```

Number of Low-Rated Products

```
SQL> SELECT COUNT(*) AS Low_Rated_Products
  2  FROM
  3  (
  4  SELECT Product_ID
  5  FROM Review
  6  GROUP BY Product_ID
  7  HAVING AVG(Rating) < 3
  8  );

LOW_RATED_PRODUCTS
------------------
                 0
```

Complete Rating

```
SQL> SELECT
  2  Rating,
  3  COUNT(*) AS Number_Of_Ratings
  4  FROM Review
  5  GROUP BY Rating
  6  ORDER BY Rating DESC;

    RATING NUMBER_OF_RATINGS
---------- -----------------
         5                 2
         4                 2
```